VWAP Workflow Implementation Comparison: Go vs. TypeScript

本文對比了 `chainlink-vwap-cre` (Go) 與 `vwap-rfq-spot` (TypeScript) 兩個專案在計算 VWAP (成交量加權平均價格) 時的實作差異。

1. 核心觀察：時間處理與兼容性

1.1 接口兼容性 (Interface Compatibility)

雖然 Go 版本內部以 15 分鐘為最小計算單位 (K 線週期)，但其對外接口設計具有高度兼容性：

- **Go 接口 (`SettlementRequest`)**：接受任意 `int64` 格式的 Unix Timestamp (秒)。
- **TS 實作**：內部寫死為 `Date.now()` 往前推 12 小時。
- **結論**：Go 版本**完全兼容** TS 所提供的任意時間戳輸入，不會因為輸入未經 15 分鐘對齊而報錯。

1.2 內部對齊邏輯 (Internal Snap-to-Grid)

Go 版本在收到請求後，會自動將時間窗口進行「內縮對齊」，以確保數據的公平性與完整性：

- **起始時間 (Start Time)**：向上取整 (Ceiling) 到下一個 15 分鐘邊界。
- **結束時間 (End Time)**：會包含涵蓋該時間點的最後一根 15 分鐘 K 線。

---

2. 架構設計與業務邏輯差異

特性	Go 實作 (`vwap-eth-quote-flow`)	TS 實作 (`cre-workflow`)	影響評估
業務耦合度	**高耦合 (OrderID)**：強制要求並寫入 OrderID。若合約不支援，則需額外封裝。	**低耦合 (通用)**：純粹輸出價格數據，不關聯特定業務物件。	TS 版本更符合目前「通用型預言機」的合約需求。
數據源數量	**5 家交易所** (Binance, OKX, Bybit, Coinbase, Bitget)	**1 家交易所** (僅限 Binance)	Go 版本在數據獲取層級具有更高的強健性與容錯率。
成交額計算	**精確數據**：直接從 API 提取 `QuoteVolume`。	**估算數據**：使用 `Typical Price * Volume` 近似計算。	TS 版本在極端波動時會產生較大誤差。
異常清洗	**多重過濾**：離群值剔除 (Median-based) 與 Flash Crash 檢查。	**無**：直接信任原始數據。	Go 版本能有效抵禦異常插針或數據餵送錯誤。
觸發機制	**按需觸發 (HTTP)**：針對特定結算請求。	**週期觸發 (Cron)**：每 12 小時執行一次。	Go 適合特定的 RFQ 結算流程，TS 適合餵價服務。
精度	**8 位小數 (1e8)**	**6 位小數 (1e6)**	需注意鏈上合約 Math 庫的位數對

			接。

---

3. 寫入邏輯分析

3.1 Go (結算導向 / 業務特定)

- **特點**：將 `OrderID` 作為鏈上寫入的必要參數。
- **風險**：如果鏈上合約 (如 `ChainlinkVWAPAdapter`) 只預期接收價格與時間，Go 的寫入邏輯將無法直接對接，需要修改 `updateReserves` 函式。
- **打包方式**：Bit Packing 雖然節省 Gas，但降低了數據的可讀性與通用性。

3.2 TS (通用導向 / 模組化)

- **特點**：採用標準 ABI Encoding，結構為 `(startTime, endTime, price)`。
- **優勢**：符合目前大多數預言機適配器 (Adapter) 的設計模式，易於整合進現有的合約架構。

---

4. 總結建議

1. **邏輯移植建議**：建議將 Go 版本的 **「多源聚合」** 與 **「異常清洗 (Outlier Scrubbing)」** 邏輯移植到 TS 或通用框架中。
2. **去耦合**：在實作時應避免像 Go 版本那樣將 `OrderID` 寫死在核心流程中，應保持價格計算與業務邏輯的分離。
3. **對齊建議**：雖然接口兼容，但建議 Client 端發送請求前先自行進行 `Round(15m)`，確保鏈上紀錄的時間區間與計算區間一致，降低審計歧義。